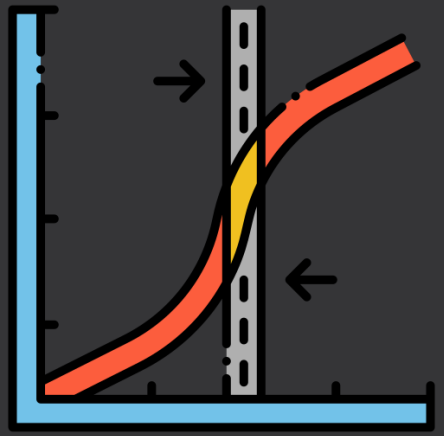


Logistic Regression

Lecture 13 – HCCDA-AI

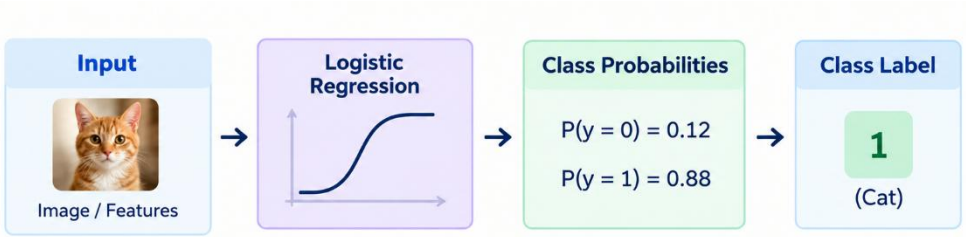
Imran Nawar

Slides adapted from Andrew Ng, Machine Learning (Stanford University / Coursera)

25 Oct 2026

Classification

- Predicts discrete-valued output.
- Classification is the task of predicting a discrete label (or class) for an input.
- Output is categorical: each input is assigned to one of the predefined classes.



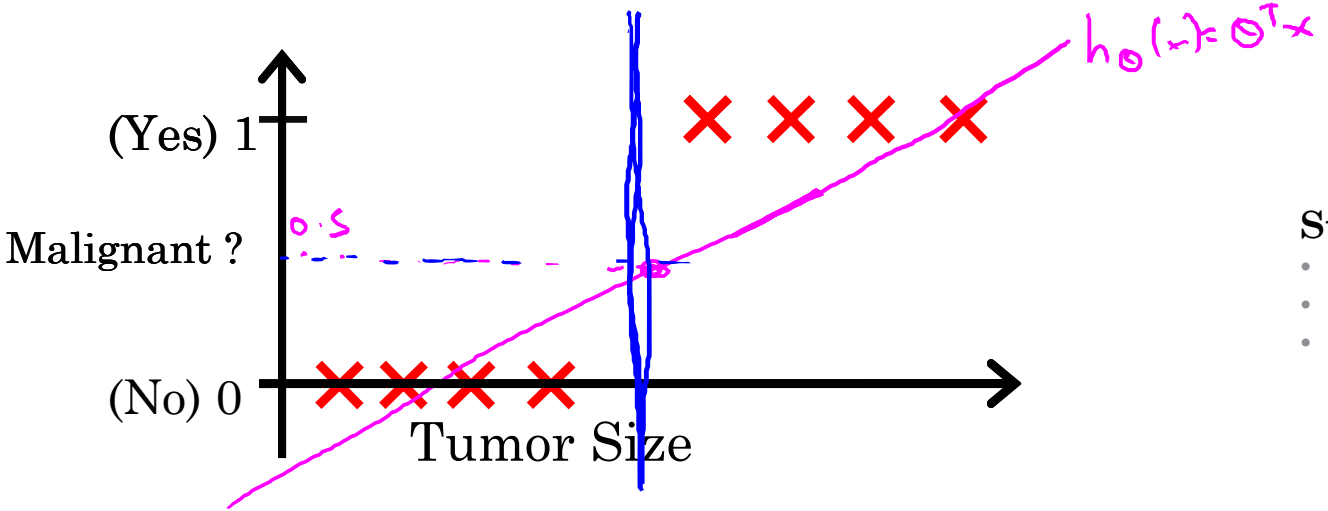
Common Examples:

- **Email:** Spam / Not Spam?
- **Online Transactions:** Fraudulent (Yes / No)?
- **Tumor:** Malignant / Benign ?

$$y \in \{0, 1\}$$

0: “Negative Class” (e.g., benign tumor)
1: “Positive Class” (e.g., malignant tumor)

Classifying Tumor as Malignant or Benign:



Steps:

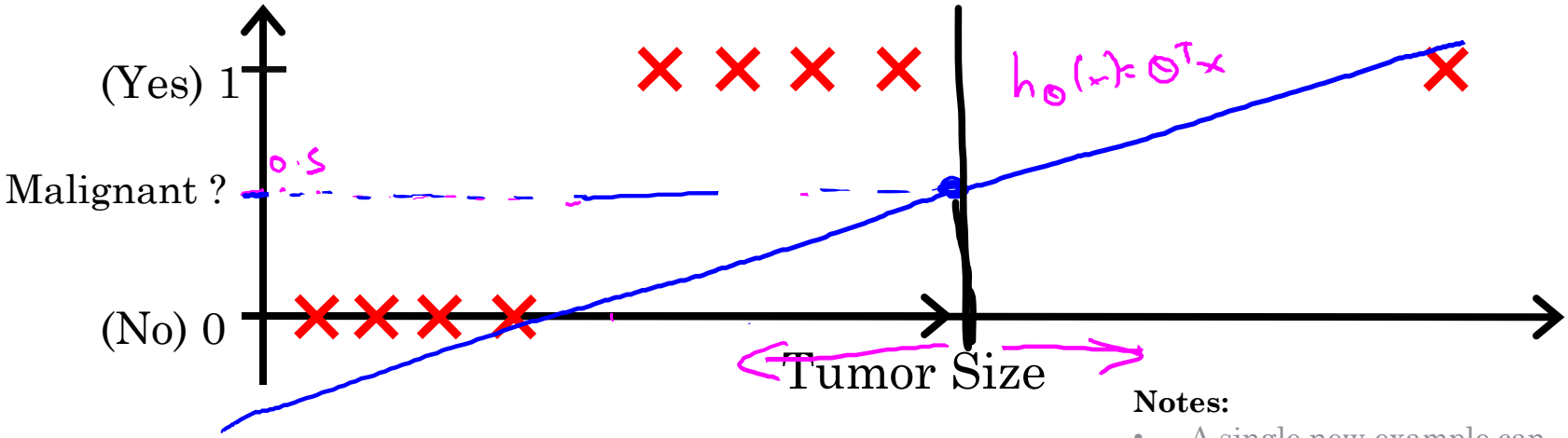
- Apply Linear Regression
- Pick a threshold
- For this dataset, linear regression seems to work reasonably well.

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"

Classifying Tumor as Malignant or Benign:



Notes:

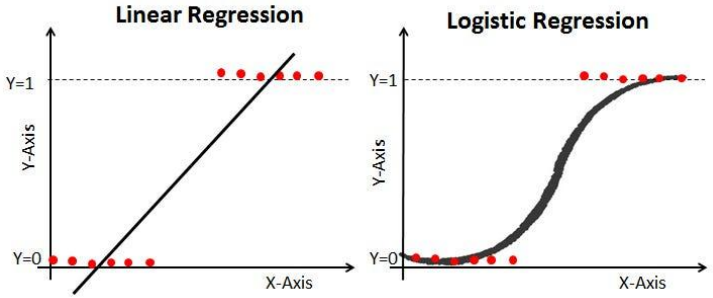
- A single new example can break it
- Applying linear regression to a classification problem usually is not a great idea!!
- Linear regression can output values below 0 or above 1, even though $y \in \{0, 1\}$.

Conclusion:

- We need a model designed to predict probabilities (between 0 and 1) and handle classification boundaries more reliably.

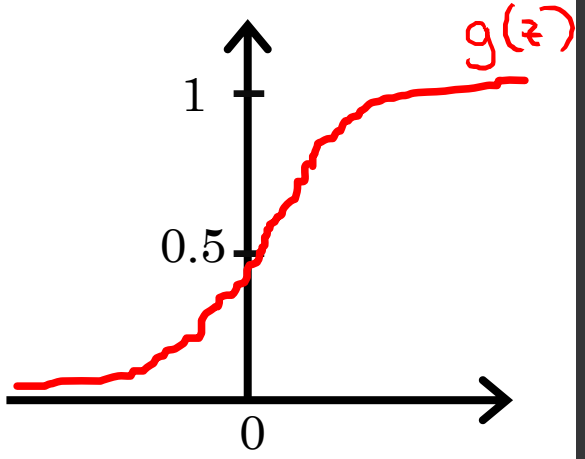
Logistic Regression

- Logistic regression predicts the probability that the input belongs to **class 1**, the class of interest.
- It uses the **sigmoid** (logistic function) to convert linear output into probabilities in the range (0, 1).



Sigmoid Function:

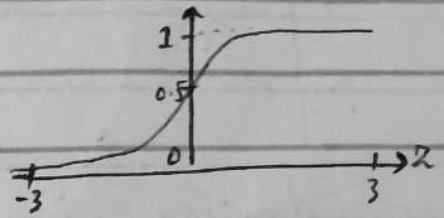
$$g(z) = \frac{1}{1 + e^{-z}}$$



- Converts linear output into probabilities.
- Helps determine class labels based on a threshold (e.g., ≥ 0.5 → class 1, < 0.5 → class 0).

Sigmoid Function:

Want outputs between 0 and 1



Output between 0 and 1

$$g(z) = \frac{1}{1+e^{-z}}$$

$$0 < g(z) < 1$$

$e = 2.71828$ mathematical constant

⇒ Linear regression: $h_{\vec{\theta}}(\vec{x}) = \theta_0 + \vec{\theta} \cdot \vec{x}$

Let's suppose $z = \theta_0 + \vec{\theta} \cdot \vec{x}$ (store in variable z)

⇒ Put value of z in ~~Logistic regression~~ Sigmoid function

$$h_{\vec{\theta}}(\vec{x}) = g(\theta_0 + \vec{\theta} \cdot \vec{x}) = \frac{1}{1 + e^{-(\theta_0 + \vec{\theta} \cdot \vec{x})}}$$

Logistic Regression

Hypothesis Representation

Logistic Regression Model

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Want $0 \le h_\theta(x) \le 1$

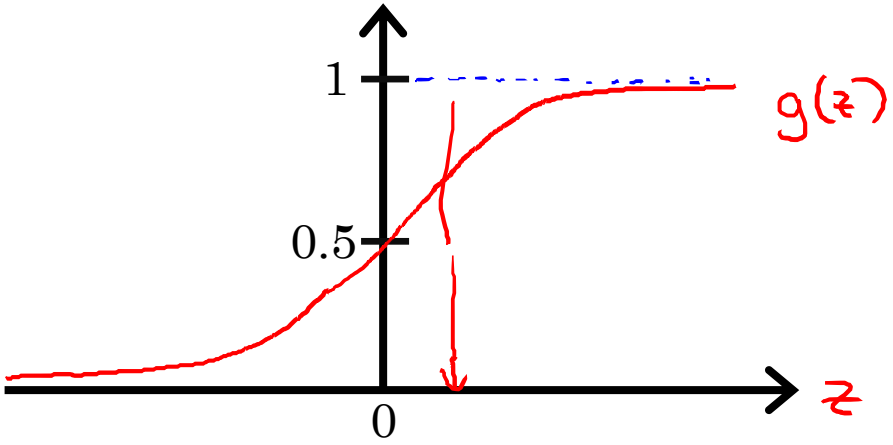
$$h_\theta(x) = g(\theta^T x)$$

where

$$g(z) = \frac{1}{1+e^{-z}}$$

Therefore

$$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$$



- *g(z)* is called the sigmoid function (also known as the logistic function).
- In neural networks, *g(z)* is also called the activation function.

Interpretation of Hypothesis Output

$h_\theta(x)$ = estimated probability that y = 1 on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_\theta(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

$h_\theta(x) = P(y=1|x;\theta)$

"probability that y = 1, given x, parameterized by θ"

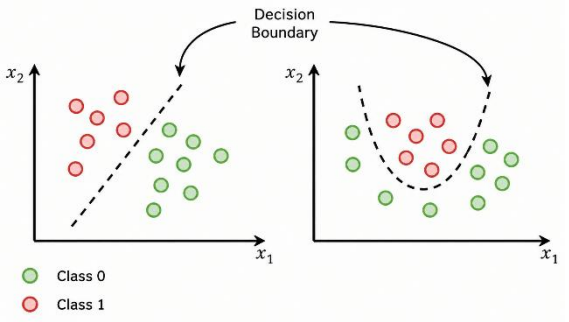
$y = 0$ or 1

$$P(y=0|x;\theta) + P(y=1|x;\theta) = 1$$
$$P(y=0|x;\theta) = 1 - P(y=1|x;\theta)$$

Logistic Regression

Decision Boundary

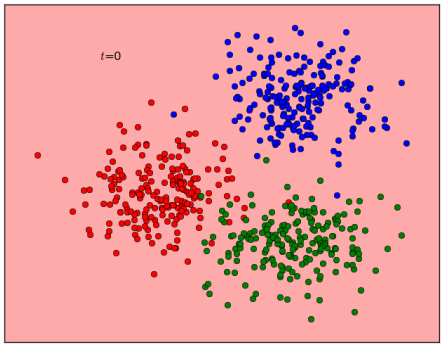
Decision Boundary



- A decision boundary is a line, surface, or hypersurface that separates different classes in a feature space.
- It guides how data points are classified in a supervised learning algorithm.

Decision Boundary in Logistic Regression:

- The boundary is the set of points where $h\theta(x) = 0.5$, which is the same as $\theta^T x = 0$ (derived on the next slide).
- Training finds the θ that places this boundary so new data are classified accurately.



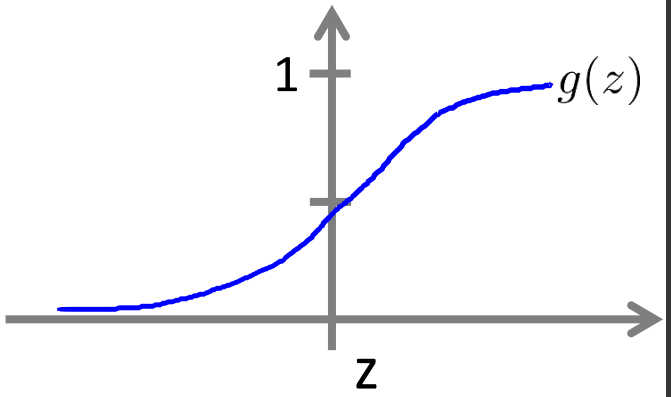
Key Insights:

- **Linear Decision Boundary:** With the original features, the boundary is linear: a line in 2D, a plane in 3D, a hyperplane in higher dimensions. It suits problems where classes can be separated by a straight line.
- **Non-Linear Boundaries:** Logistic regression can learn these by adding polynomial features (see the non-linear boundaries slide). Decision trees, SVMs, and neural networks also handle non-linear separations.

Logistic Regression

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$



predict " $y = 1$" if $h_\theta(x) \geq 0.5$

$\theta^T x \geq 0$

$g(z) \geq 0.5$ when $z \geq 0$

$h_\theta(x) = g(\theta^T x$

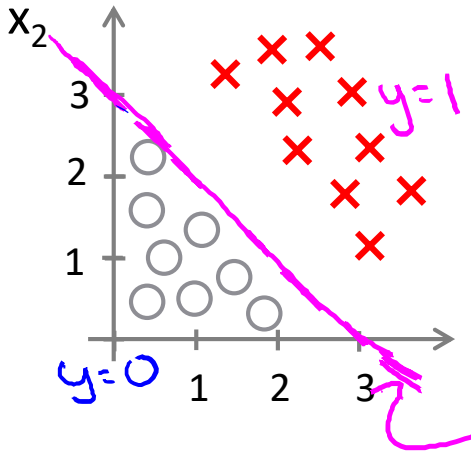
predict " $y = 0$" if $h_\theta(x) < 0.5$

$\theta^T x < 0$

$g(z) < 0.5$ when $z < 0$

Decision Boundary

$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

-3 1 1

Decision boundary

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$

$\theta^T x$

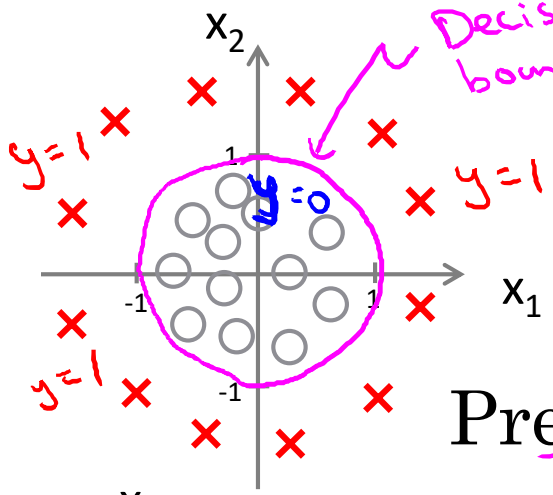
$x_1 + x_2 = 3$

$x_1 + x_2 \geq 3$

$x_1 + x_2 < 3$

$y = 0$

Non-linear decision boundaries



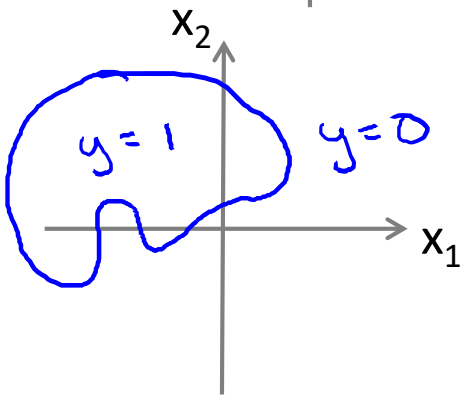
$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$

Predict "$y = 1$" if $-1 + x_1^2 + x_2^2 \geq 0$

$x_1^2 + x_2^2 = 1$

$x_1^2 + x_2^2 \geq 1$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \dots)$$

Logistic Regression

Cost Function

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ?

Cost Function

Linear Regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Cost of a single example: $\text{Cost}(h_\theta(x^{(i)}), y^{(i)}) = \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Therefore: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} Cost(\, h_\theta(x^{(i)}), y^{(i)})$

For logistic regression: $h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$

Squared error with the sigmoid hypothesis leads to a non-convex cost function.

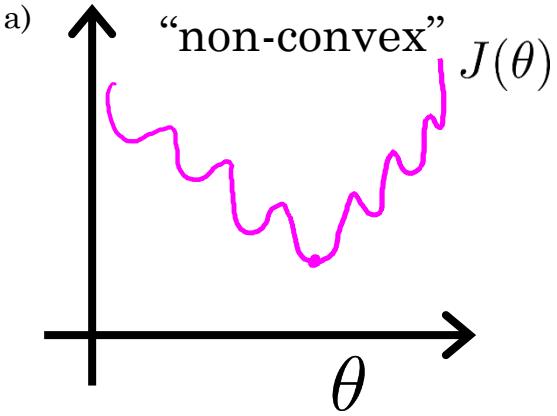
Gradient descent may get stuck in local minima.

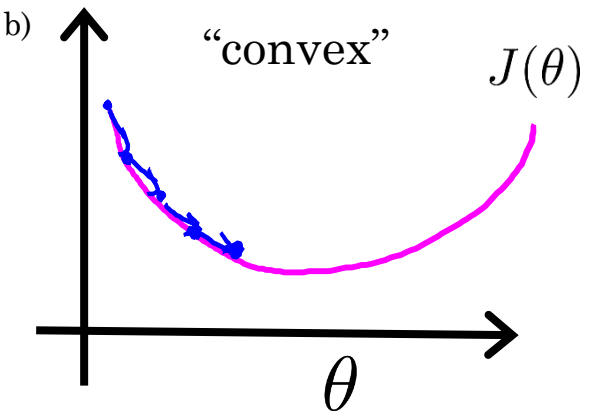
Cost Function

- The squared-error cost works well for linear regression, but if we use it for logistic regression, J(θ) becomes a non-convex function of the parameters θ.
- For logistic regression:

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

- If we put this sigmoid function into the cost function. The $J(\theta)$ will look like figure (a), with many local minima.
- Figure (a) is non-convex, gradient descent is not guaranteed to converge to the global minimum.
- Figure (b) is convex, gradient descent reaches the global minimum, which is what we want.

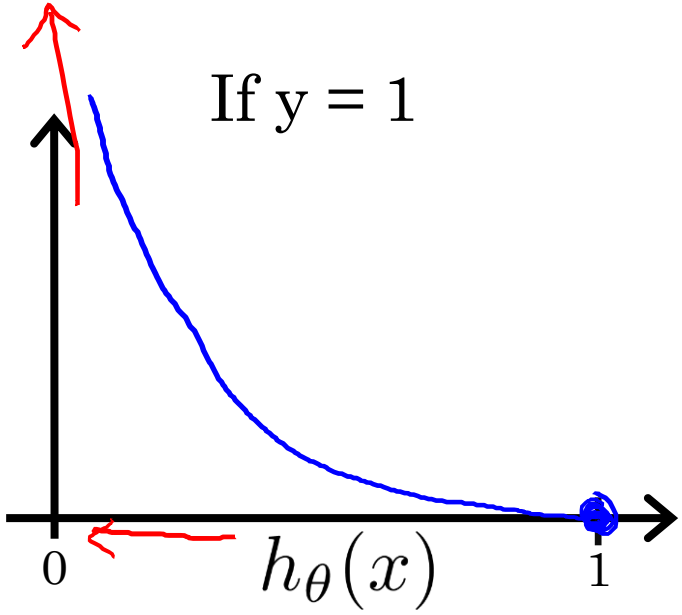




Logistic Regression Cost Function

Binary Cross Entropy Loss

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$



Cost $= 0$ if $y = 1, h_\theta(x) = 1$
But as $h_\theta(x) \to 0$
$Cost \to \infty$

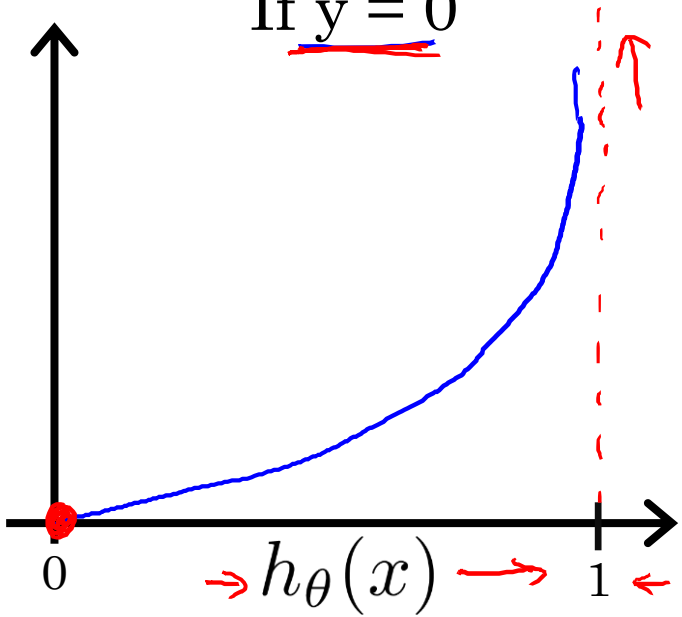
Captures intuition that if $h_\theta(x) = 0$, (predict $P(y = 1|x; \theta) = 0$), but $y = 1$, we'll penalize learning algorithm by a very large cost.

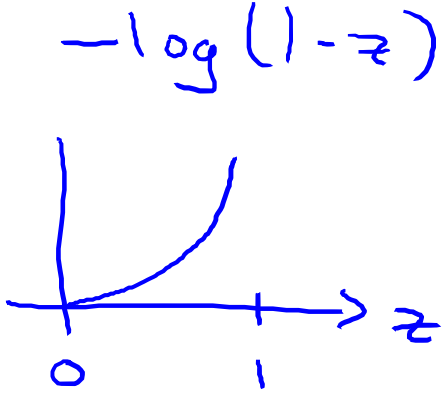
- Loss is lowest when $h_\theta(x^{(i)})$ predicts close to true $y^{(i)}$ label

Logistic Regression Cost Function

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

If y = 0





Logistic Regression

Simplified Cost Function and Gradient Descent

Overall cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

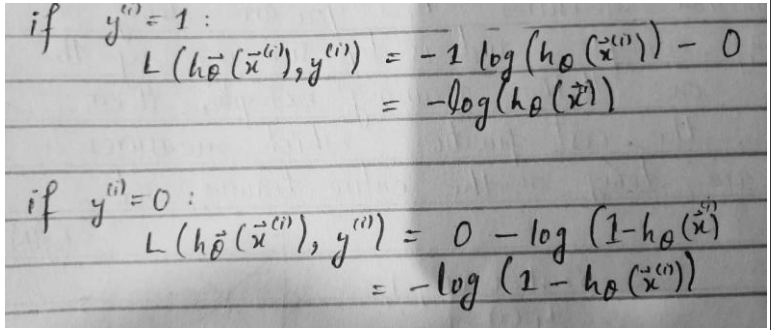
Cost (loss) of a single training example

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

$$\text{Cost}(h_\theta(x), y) = -y \log(h_\theta(x)) - (1-y) \log(1 - h_\theta(x))$$

This equation is completely equivalent to the above more complex formula.

Note: $y = 0$ or 1 always



Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

To fit parameters : θ

$$\min_\theta J(\theta)$$

To make a prediction given new x:

Output $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

(simultaneously update all θ_j)

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat{

$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

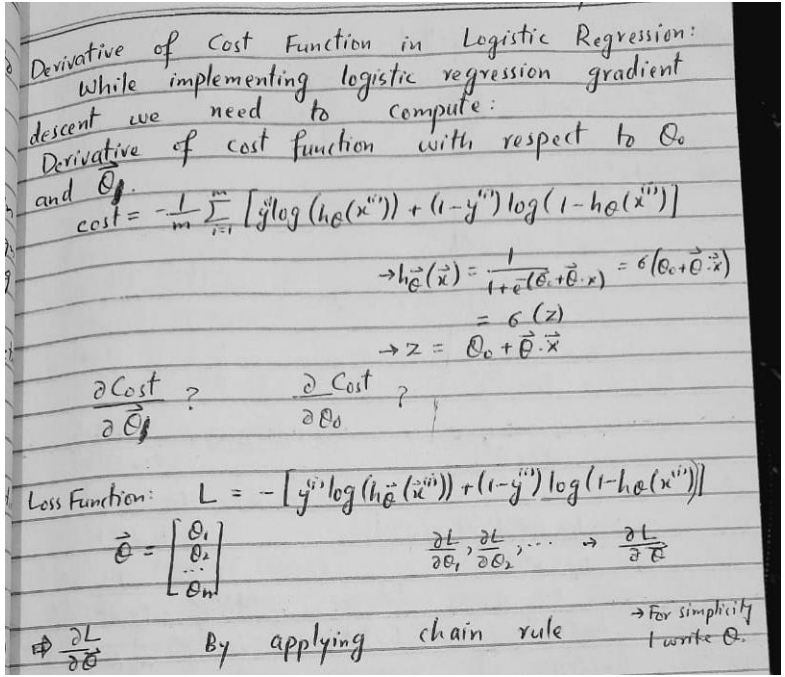
$h_\theta(x) = \theta^T x$

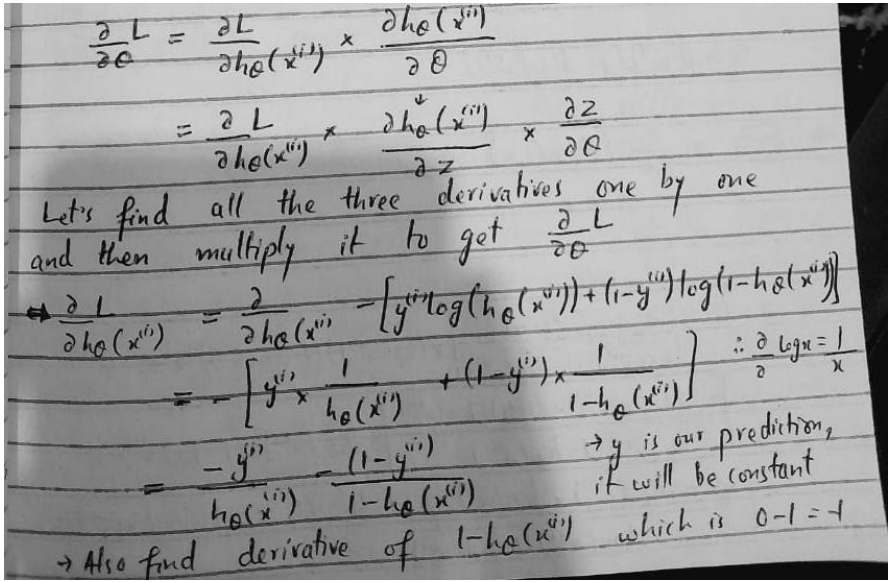
$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$

Algorithm looks identical to linear regression!

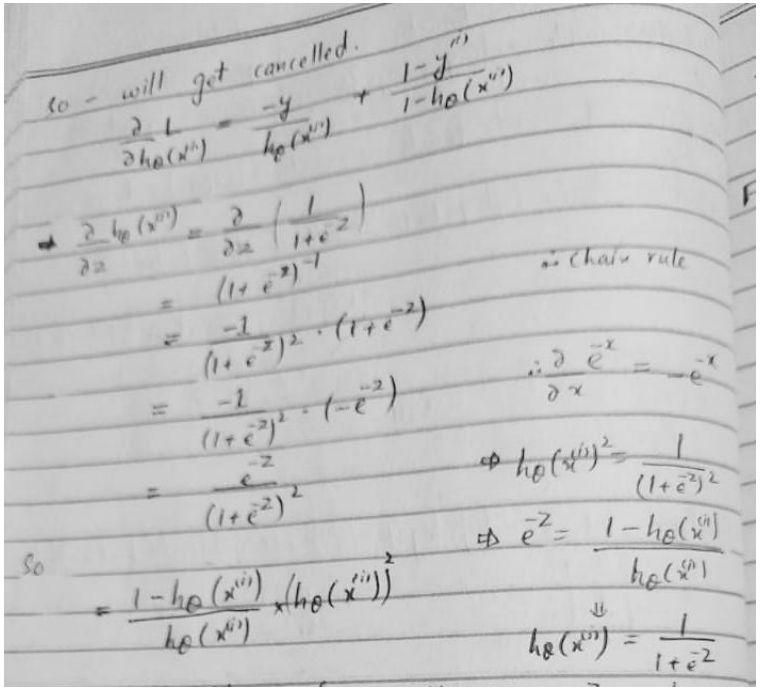
But the hypothesis $h_\theta(x) = \frac{1}{(1 + e^{(-\theta^T X)})}$ is different!

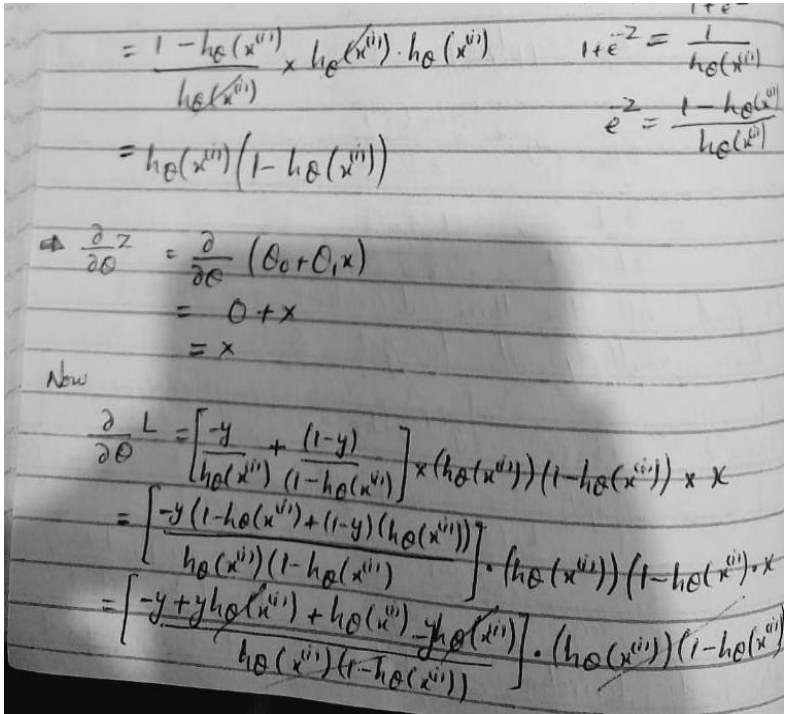
Gradient of Binary Cross Entropy Loss Function



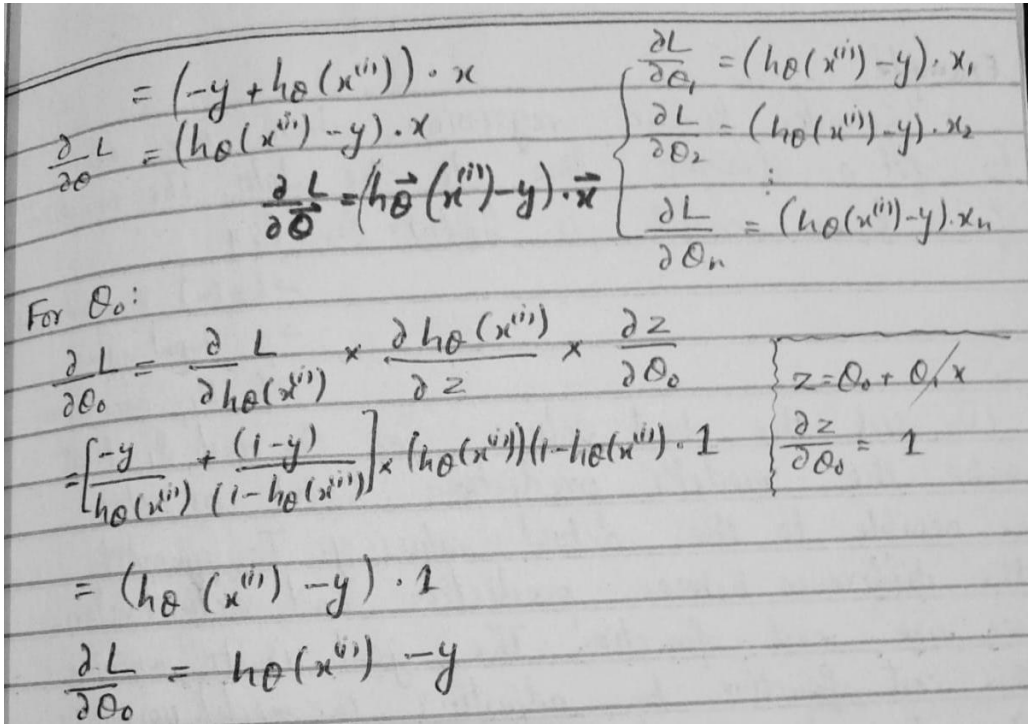


Gradient of Binary Cross Entropy Loss Function





Gradient of Binary Cross Entropy Loss Function



Logistic Regression

Advanced Optimization

Optimization algorithm

Cost function $J(\theta)$. Want $\min_\theta J(\theta)$.

Given θ , we have code that can compute

- $J(\theta)$
- $\frac{\partial}{\partial \theta_j} J(\theta)$ (for $j = 0, 1, \ldots, n$)

Any optimization algorithm needs only these two computations. Gradient descent is one way to use them.

Gradient descent:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$\}$ (simultaneously update all θ_j)

Advanced Optimization algorithms

Optimization algorithms:

- Gradient descent, Conjugate gradient, BFGS, L-BFGS

Advantages (Conjugate gradient, BFGS, L-BFGS**):**

- No need to manually pick α
- Often faster than gradient descent.

Disadvantages:

- More complex
- BFGS and L-BFGS are quasi-Newton methods: they approximate the Hessian instead of computing it

Logistic Regression

Newton's Method

Newton's Method

Newton's Method is a second-order optimization algorithm that uses both the gradient and the Hessian matrix to find a minimum of the cost function.

Key Points:

- Uses **second-order derivatives** (Hessian matrix) for optimization.
- Needs fewer iterations than Gradient Descent, but each iteration is computationally expensive (computing and inverting H).
- Used in logistic regression, SVMs, neural networks, and nonlinear optimization.

Formula:

$$w = w - H^{-1} \nabla J(w)$$

where:

- H = Hessian matrix (second derivatives of the cost function J(θ).
- $\nabla J(w)$= gradient of the cost function (vector of first derivatives).

Newton's Method vs. Gradient Descent

Feature	Newton's Method	Gradient Descent
Uses	Second-order derivatives	First-order derivatives
Convergence	Fewer iterations (faster)	More iterations (slower)
Computational Cost (per iteration)	Higher (Hessian inversion)	Lower
Step Size Adjustment	Not required	Requires tuning
Suitable for Large Datasets	No	Yes

Applications:

- **Logistic Regression** (Newton-Raphson method)
- **Support Vector Machines (SVMs)**
- **Neural Network Training** (L-BFGS method)

Key Takeaway:

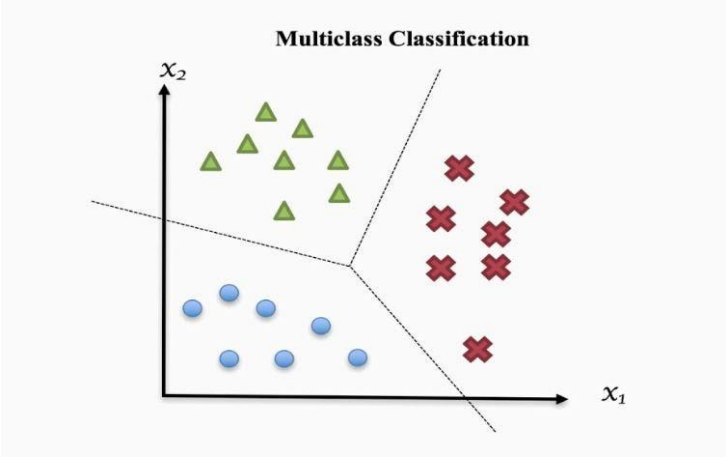
- Newton's Method needs fewer iterations but is computationally more expensive per iteration than Gradient Descent. Suitable for small to medium-sized problems (few features) where quick convergence is needed.

Logistic Regression

Multi-class classification: One-vs-all

Multiclass classification

- Multi-class classification is the process of assigning each data instance to **one** class among **three or more** possible categories. Unlike binary classification, where there are only two classes, multi-class classification deals with **multiple** distinct labels.



Examples

- Classifying emails as Spam, Promotions, or Primary.
- Identifying handwritten digits (0-9)
- Categorizing images into Dogs, Cats, and Birds.

Common Algorithms:

- Logistic Regression (One-vs-Rest, Softmax Regression)
- Decision Trees & Random Forest
- Support Vector Machines (SVM, One-vs-One strategy)
- Neural Networks (Deep Learning models like CNNs for image classification)

Loss Function:

- Categorical cross-entropy (often called softmax loss) is commonly used with softmax regression and neural networks. It measures how far the predicted class probabilities are from the true labels.

Multiclass classification

Email foldering/tagging: Work, Friends, Family, Hobby

$y=1$ $y=2$ $y=3$ $y=4$

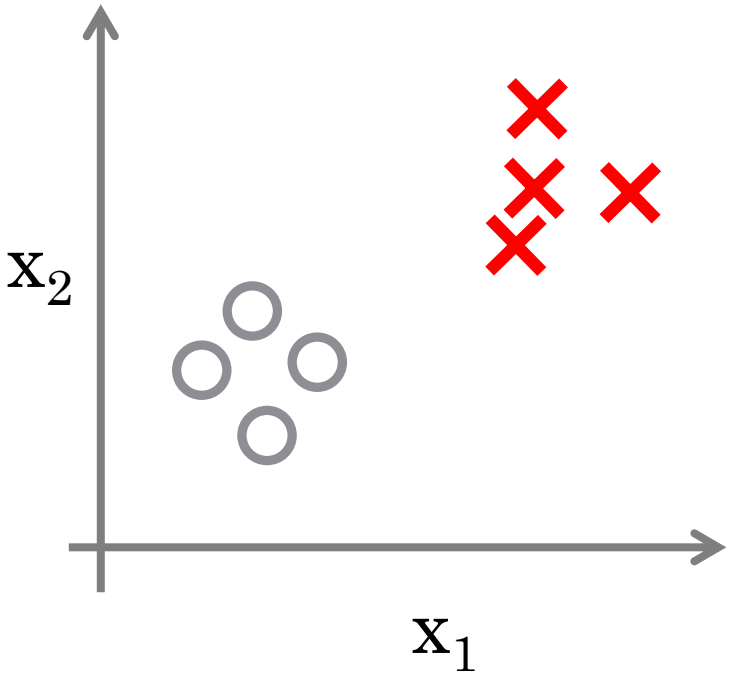
Medical diagnosis: Not ill, Cold, Flu

$y=1$ 2 3

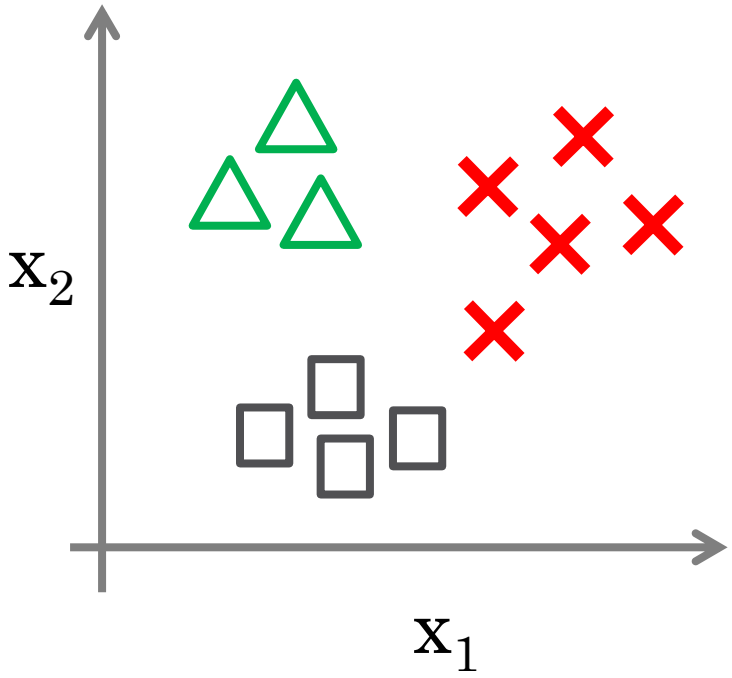
Weather: Sunny, Cloudy, Rain, Snow

$y=1$ 2 3 4

Binary classification:

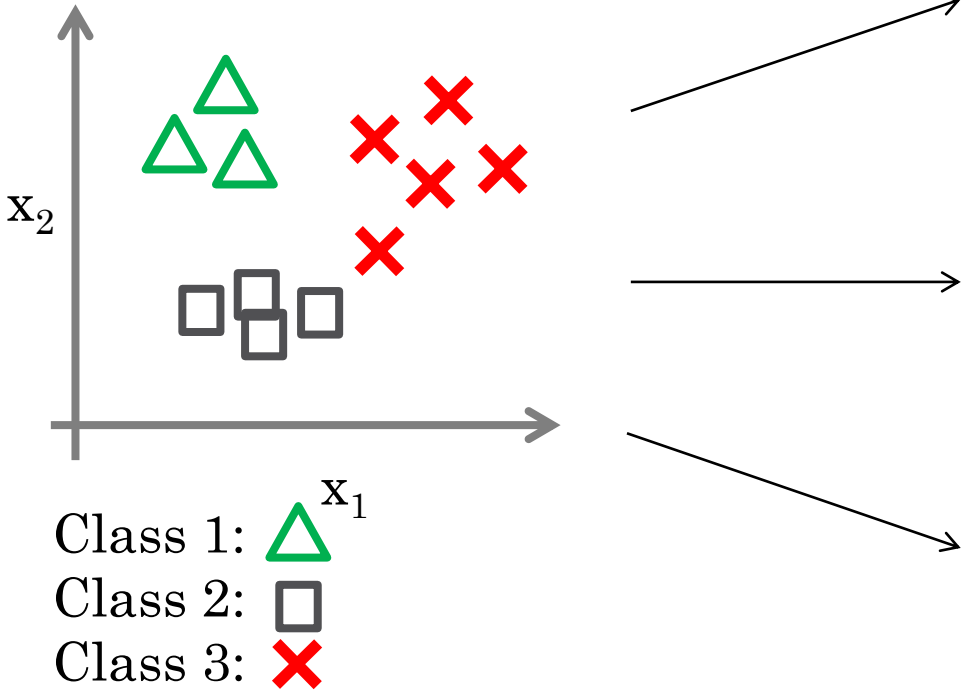


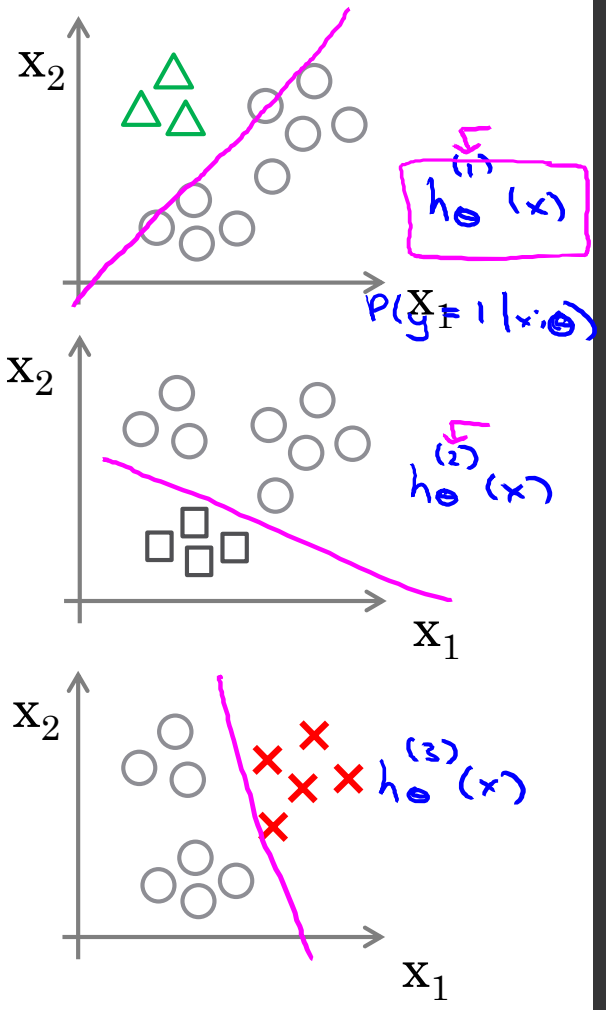
Multi-class classification:



One-vs-all (one-vs-rest):

One-vs-all is a strategy used in multiclass classification where a separate binary classification model is trained for each class.





$$h_\theta^{(i)}(x) = P(y = i|x; \theta) \qquad (i = 1, 2, 3)$$

Thank You